Manual de ejecución de la tarea 3

Clasificación binaria con Apache Spark y PostgreSQL

Adrian Rojas Rivera | Cédula 117550336 | Modalidad individual
Módulo 4 Big Data | 23 de octubre de 2026

Este manual permite reproducir y calificar el notebook de predicción de abandono de clientes. El paquete incluye los datos, el código, las salidas y dos servicios Docker: Spark con Jupyter y PostgreSQL. La ejecución completa crea las tres tablas requeridas y compara dos clasificadores sobre la misma reserva final.

1 Preparar el entorno

Instale e inicie Docker Desktop con Docker Compose v2. Se recomienda asignar al menos 4 GB de memoria y dos CPU a Docker, y cerrar otros contenedores pesados. La primera construcción requiere Internet para descargar imágenes y dependencias. No se necesita instalar Python, Java ni PostgreSQL en el equipo.

Descomprima el ZIP y abra una terminal dentro de la carpeta T3. Deben verse Dockerfile, docker-compose.yaml, data y notebooks. Todos los comandos de este manual se ejecutan desde esa carpeta en la terminal del equipo.

```
docker compose up -d --build
docker compose ps
```

Espere a que postgres aparezca healthy y bigdata como Up. La imagen usa Java 17, Python 3.10, Spark 3.5.1 y PostgreSQL JDBC 42.7.3. La base usa PostgreSQL 16.6. Las dependencias de Python se fijan en requirements.txt.

Abrir Jupyter

```
http://127.0.0.1:8893/lab?token=tarea3-local
```

Abra notebooks/Tarea_3.ipynb. El notebook entregado contiene resultados guardados para facilitar la revisión. Para reproducirlo, siga la página siguiente. El token predeterminado de acceso es tarea3-local.

La primera descarga depende de la conexión. La duración del procesamiento se registra en output/Resumen_ejecucion.json; no incluye la construcción de la imagen.

2 Ejecutar el notebook y consultar las tablas

En Jupyter use Kernel > Restart Kernel and Run All Cells. Espere el mensaje Ejecución terminada y tablas verificadas y guarde el notebook. Como alternativa, este comando abre un kernel nuevo, ejecuta el mismo archivo y guarda sus salidas automáticamente:

```
docker compose exec -T bigdata python scripts/ejecutar_notebook.py
```

Use una sola modalidad a la vez. Las celdas deben ejecutarse en orden. El procesamiento termina su sesión Spark al finalizar. Los CSV de resultados, figuras, particiones y pipelines ajustados se regeneran en output.

Consultar PostgreSQL desde el equipo

Después de la ejecución, compruebe conteos, tipos, correspondencia de atributos y matrices de confusión con las consultas incluidas:

```
docker compose exec -T postgres psql -U estudiante -d tarea3 \
  -f /dev/stdin < sql/Verificacion.sql
```

En PowerShell puede usar esta alternativa de lectura del archivo:

```
Get-Content sql/Verificacion.sql |
  docker compose exec -T postgres psql -U estudiante -d tarea3
```

En PowerShell, el salto de línea después del signo de tubería continúa el mismo comando.

Tabla	Filas esperadas	Contenido
tarea3	10000	Siete columnas originales depuradas
modelo1	1921	Reserva final y prediccion de regresión logística
modelo2	1921	Reserva final y prediccion de bosque aleatorio

Las tablas de predicción conservan id, edad, gasto_mensual, tickets_soporte, meses_cliente, usa_app y etiqueta, y añaden prediccion entera. Contienen las filas de reserva, no todas las de tarea3. Los indicadores y conteos son enteros; id es texto y gasto_mensual es double precision.

PostgreSQL usa host interno postgres, puerto 5432, base tarea3, usuario estudiante y clave local tarea3_local. No se publica un puerto SQL al equipo. El usuario y la base los crea el contenedor; Spark crea las tablas mediante JDBC con overwrite.

3 Datos y preparación

El archivo data/dataset.csv es el ejemplo de abandono suministrado en L5/T3.zip. Su identidad se verifica con el SHA-256 de data/Fuente.json. El material no documenta una población real, el mecanismo de generación ni la moneda del gasto. El análisis se limita a este archivo.

Campo	Tipo Spark	Significado
id	string	Identificador del registro; se excluye de los predictores
edad	integer	Edad en años
gasto_mensual	double	Gasto mensual; moneda no especificada
tickets_soporte	integer	Número de solicitudes de soporte
meses_cliente	integer	Antigüedad en meses
usa_app	integer	Indicador binario de uso de la aplicación
etiqueta	integer	Objetivo: 0 permanece, 1 abandona

Reglas de calidad

Se declara StructType explícito, sin inferSchema. Se recortan espacios en ID y se rechazan nulos obligatorios, ID vacío, registros corruptos, edad no positiva, valores negativos en gasto, tickets o antigüedad, gasto no finito e indicadores fuera de 0 y 1. Se eliminan duplicados exactos. Si un ID tiene versiones válidas diferentes, se excluyen todas para evitar elegir arbitrariamente un registro.

La ejecución encontró 10000 filas válidas, sin duplicados exactos ni identificadores conflictivos. Las reglas se ejercitan también con datos de prueba que sí contienen errores. Que no se rechacen filas no demuestra ausencia de errores en la fuente.

Recorrido y exploración

Los datos depurados se escriben en tarea3 y se vuelven a leer desde PostgreSQL para continuar. La igualdad de todas las filas se verifica en ambas direcciones. Un hash del ID con semilla 27 separa 8079 registros de entrenamiento y 1921 de reserva final; no depende del orden de lectura JDBC.

La exploración usa solo entrenamiento. Incluye distribución de clases, estadísticas, histogramas y correlaciones de Pearson calculadas con Spark. Los abandonos representan 41,03% del entrenamiento. Las correlaciones con etiqueta más marcadas son tickets_soporte, 0,295, y gasto_mensual, 0,177. Son asociaciones, no evidencia causal.

4 Entrenamiento y validación cruzada

Cada modelo usa un Pipeline de VectorAssembler, StandardScaler y clasificador. Se incluyen cinco predictores; ID y etiqueta no entran al vector de características. StandardScaler centra y divide por la desviación estándar. En regresión logística se desactiva la estandarización interna para usar la etapa explícita.

El escalador se ajusta dentro de cada fold. De esa forma, la media y desviación no se estiman con el fold que se está validando. El bosque sigue el mismo recorrido, aunque sus particiones por umbrales no requieren escalado por el mismo motivo que la regresión regularizada. PostgreSQL conserva los atributos originales en columnas, sin vectores.

Protocolo

Se construyen cuatro folds equilibrados por clase dentro del entrenamiento. CrossValidator recibe esa columna mediante foldCol. Cada candidato se valida una vez en cada fold; después se reajusta el mejor pipeline con todo el entrenamiento. Se usa la media de AUC ROC para seleccionar parámetros. La reserva final no interviene en la búsqueda ni en el escalado.

Modelo	Búsqueda	Selección obtenida
Regresión logística	regParam: 0,01 o 0,1 elasticNetParam: 0 o 0,5	regParam 0,01 elasticNetParam 0,5
Bosque aleatorio	maxDepth: 4 u 8 minInstancesPerNode: 2 o 10 40 árboles	maxDepth 4 minInstancesPerNode 2

Hay cuatro candidatos por modelo y cuatro folds: 32 ajustes de validación cruzada y dos reajustes finales. El valor K=4 es una decisión de esta solución, coherente con el ejemplo explicado en L5; el enunciado no fija ese número.

Medida AUC ROC	Regresión	Bosque
Entrenamiento	0.7009	0.7115
Media de CV	0.7003	0.6997
Desviación entre folds	0.0036	0.0086

La diferencia entre entrenamiento y CV es mayor en el bosque, lo que sugiere más optimismo de ajuste. Estas métricas se muestran separadas de la evaluación final. El umbral de decisión se conserva en 0,5 para ambos modelos.

5 Resultados sobre la reserva final

Se evalúan las mismas 1921 observaciones para ambos modelos. La clase positiva es abandono. Precision indica la proporción correcta entre alertas de abandono; recall, la fracción de abandonos detectados. F1 corresponde a la clase positiva, no al promedio ponderado entre clases.

Métrica	Regresión	Bosque
AUC ROC	0.7181	0.7196
AUC PR	0.6242	0.6299
Accuracy	0.6710	0.6653
Precision positiva	0.6381	0.6564
Recall positivo	0.4988	0.4243
F1 positivo	0.5599	0.5154

Resultado	Regresión	Bosque
Permanencias correctas	887	936
Falsas alertas de abandono	228	179
Abandonos sin detectar	404	464
Abandonos detectados	402	342

Predecir siempre la clase mayoritaria del entrenamiento alcanza accuracy 0,5804 en reserva. Ambos modelos superan esa referencia. La AUC PR se contrasta también con la proporción positiva de reserva, 0,4196. Las curvas dibujadas en el notebook agrupan puntuaciones por intervalos; las métricas numéricas las calcula el evaluador de Spark.

El bosque tiene una AUC ROC ligeramente superior, con diferencia de 0,0016, y genera 49 falsas alertas menos. La regresión detecta 60 abandonos adicionales y obtiene mayor recall y F1. Si se prioriza detectar abandonos, la regresión sería la opción a considerar; si el contacto innecesario es costoso, el contraste favorece la precisión del bosque. No hay costos de negocio para resolver esa elección de forma definitiva.

Ninguno detecta más de la mitad de los abandonos con el umbral fijado. La diferencia pequeña de AUC no prueba superioridad universal ni significación estadística. No se reajusta el umbral ni se amplía la búsqueda tras revisar la reserva. La fuente no permite evaluar generalización temporal o a otra población.

6 Verificación mantenimiento y fuentes

Comprobaciones realizadas

La ejecución del 23 de octubre de 2026 terminó sin errores de celda y conservó cinco figuras en el notebook. Pasaron cinco pruebas automatizadas. El informe output/Pruebas.xml y las consultas output/Verificacion_SQL.txt acompañan los resultados.

```
docker compose exec -T bigdata pytest -q --junitxml=output/Pruebas.xml
```

El notebook comprueba ausencia de solapamiento entre entrenamiento y reserva, folds por clase, igualdad de contenido tras JDBC, columnas sin vectores y predicciones válidas. Repite las tres escrituras con overwrite y verifica que los datos y conteos se mantengan. El resultado está en output/Resumen_ejecucion.json y doc/Validacion.md.

Detener o resolver fallos

```
docker compose down
docker compose up -d
```

El primer comando detiene los servicios conservando el volumen de PostgreSQL; el segundo los vuelve a iniciar. Los resultados del notebook se conservan en la carpeta local. down -v elimina la base de este entorno y exige recrear sus tablas mediante el notebook.

Si Docker no responde, inicie Docker Desktop. Si 8893 está ocupado, cambie el puerto del equipo en compose y recree el servicio. Para un fallo de conexión, revise que postgres esté healthy y ejecute desde bigdata. Si el kernel termina por memoria, cierre otros contenedores y aumente los recursos de Docker. El README amplía estas instrucciones y explica la configuración opcional de .env.

```
docker compose logs --tail=80 bigdata postgres
```

Fuentes del procedimiento

Enunciado Tarea 3 Spark ML y paquete L5/T3.zip: requisitos, ejemplo Binary classification.ipynb, CSV y entorno. L5, video1324764996.mp4, 00:00-00:11:10: esquema, escalado, persistencia tabular, K-fold, evidencias y overwrite. L4, video1908915064.mp4, 00:28:30-00:38:30 y 01:04:30-01:07:30: conectores, DataFrames y Spark ML. Los intervalos son aproximados.

Documentación complementaria de Apache Spark 3.5.1: ML Tuning, StandardScaler y JDBC Data Source. Las referencias completas y enlaces se incluyen en doc/Fuentes.md. El notebook contiene la interpretación completa de los resultados y sus limitaciones.